

Plano de Automação Completa de Qualidade

Testes automáticos de todas as ferramentas, interface, fluxos, downloads, logs e diagnóstico

Decisão principal

A implementação foi dividida em 12 lotes. O desenho recomendado utiliza um motor genérico com Playwright, cenários declarados por ferramenta, instrumentação permanente no ambiente E2E e relatórios com evidências suficientes para reproduzir cada falha.

Projeto analisado	Prazzu Tools - Laravel 12
Escopo atual	32 ferramentas e crescimento contínuo
Objetivo	Eliminar a dependência de testes manuais repetitivos
Quantidade de lotes	12 lotes

1. Resumo executivo

O projeto precisa de uma plataforma interna de testes, e não apenas de alguns scripts isolados. Essa plataforma deverá descobrir automaticamente todas as ferramentas registradas, abrir cada uma em um navegador real, preencher campos, interagir com todos os componentes relevantes, executar cenários válidos e inválidos, validar resultados, testar downloads e reunir evidências técnicas quando alguma etapa falhar.

A solução precisa crescer junto com o catálogo. Quando a ferramenta número 33, 50 ou 100 for adicionada, ela deverá entrar automaticamente na bateria de qualidade ao declarar seus cenários e seguir o contrato visual da plataforma.

Princípio central

O robô não deve editar o código para inserir e remover logs. A aplicação deve possuir uma instrumentação de teste permanente, habilitada apenas no ambiente E2E, com correlação por execução e cenário.

2. Resultado esperado

- Abrir automaticamente todas as ferramentas cadastradas.
- Validar carregamento, formulários, botões, abas, modais, links, mensagens e resultados.
- Preencher dados válidos, inválidos e casos de borda.
- Testar visitantes, usuários autenticados, planos pagos e administração quando aplicável.
- Confirmar que downloads realmente funcionam e que o arquivo gerado é válido.
- Capturar erros PHP, Laravel, JavaScript, HTTP, banco de dados e filas.
- Gerar relatório HTML e JSON, com screenshots, traces, vídeos e arquivos baixados.
- Executar testes rápidos em commits e testes completos antes de releases.
- Impedir que novas ferramentas sejam entregues sem cobertura mínima.

3. Arquitetura proposta

A arquitetura recomendada combina PHPUnit para regras e integrações com Playwright para testes no navegador. O catálogo de ferramentas será a fonte de descoberta; cada módulo fornecerá cenários específicos, enquanto um executor central fará a navegação, preenchimento, cliques, validações, downloads e coleta de evidências.

Camada	Responsabilidade	Tecnologia sugerida
Testes de domínio	Cálculos, regras normativas, validações e value objects	PHPUnit
Testes de integração	Rotas, banco, autenticação, histórico, exportadores e APIs	PHPUnit / Laravel
Testes de navegador	Formulários, cliques, UI, fluxos e downloads	Playwright
Instrumentação	Correlação de logs, exceções, queries, eventos e métricas	Laravel middleware + logging
Relatórios	Resumo, falhas, evidências e reprodução	Playwright HTML + relatório próprio
Orquestração	Execução local, paralela e em CI	Composer, npm e pipeline CI

4. O que será testado em cada ferramenta

Carregamento e integridade

- Status HTTP, ausência de 404/500 e presença do conteúdo principal.
- Assets, chamadas internas e ausência de erros de console.
- Tempo máximo de carregamento e detecção de tela vazia.

Formulários

- Campos obrigatórios, máscaras, limites, datas, moedas, selects, checkboxes e dependências entre campos.
- Envio normal, envio repetido e prevenção de duplo clique.
- Mensagens de validação e preservação dos dados quando ocorre erro.

Resultados

- Exibição do bloco de resultado e dos valores esperados.
- Conferência com casos dourados ou expectativas declaradas.
- Persistência, histórico e auditoria quando aplicáveis.

Elementos da tela

- Botões, links, abas, modais, acordeões, menus, tooltips e ações secundárias relevantes.
- Detecção de botão sem ação, elemento invisível ou erro após interação.

Downloads

- PDF, XLSX, CSV, DOCX e ZIP, conforme a ferramenta.
- Nome, extensão, MIME type, tamanho mínimo, assinatura e estrutura interna.
- Detecção de página HTML de erro salva com extensão de arquivo.

Acesso e segurança funcional

- Visitante, usuário gratuito, usuário Plus e administrador.
- Bloqueios de interface e proteção direta dos endpoints.
- CSRF, throttling e autorização nos fluxos relevantes.

Responsividade e compatibilidade

- Desktop como bateria principal.
- Subconjunto crítico em tablet e celular.
- Chrome obrigatório; Firefox e WebKit na bateria completa.

5. Contrato de teste por ferramenta

A automação não consegue deduzir sozinha quais dados são fiscal, contábil ou juridicamente corretos. Cada módulo deverá declarar cenários de teste por meio de um contrato padronizado.

- Identificador e nome do cenário.
- Perfil de acesso necessário.
- Dados que devem ser preenchidos.

- Ações que devem ser realizadas.
- Resultado esperado.
- Mensagens esperadas ou proibidas.
- Downloads esperados.
- Registros de banco ou histórico esperados.
- Nível de risco e grupo de execução.

6. Instrumentação e diagnóstico

Cada execução e cenário receberá um identificador de correlação. Esse identificador deverá acompanhar as requisições do navegador e ser incluído nos logs da aplicação. Quando houver falha, o relatório poderá reunir somente os eventos correspondentes àquele teste.

- Middleware para receber ou criar X-E2E-Run-Id e X-E2E-Scenario-Id.
- Canal de log JSON exclusivo do ambiente E2E.
- Captura centralizada de exceções com arquivo, linha e stack trace sanitizado.
- Registro opcional de queries lentas, eventos de domínio, filas e downloads.
- Captura do console, page errors, requests e responses do navegador.
- Sanitização de senhas, tokens, documentos e dados pessoais no relatório.
- Política de retenção para remover artefatos antigos automaticamente.

7. Artefatos gerados

- Relatório HTML navegável.
- Resumo JSON para integração com CI.
- Screenshot no ponto da falha.
- Trace do Playwright para reprodução passo a passo.
- Vídeo apenas quando houver falha ou quando explicitamente habilitado.
- Logs Laravel correlacionados.
- Lista de respostas HTTP com erro.
- Arquivos baixados e resultado da validação interna.
- Dump sanitizado do cenário ou snapshot do banco quando necessário.

8. Estratégia de execução

Perfil	Quando	Conteúdo	Objetivo
Smoke	A cada commit	Arquitetura, unitários, build e um cenário principal de todas as ferramentas	Feedback rápido
Pull request	Antes de integrar alterações	Cenários válidos, inválidos essenciais, acessos e downloads principais	Detectar regressões
Completo	Agendado ou pré-release	Todos os cenários, navegadores, responsividade, downloads e fluxos autenticados	Homologação técnica
Exploratório	Sob demanda ou agendado	Variações, limites, entradas inesperadas e cliques adicionais	Encontrar falhas não previstas

9. Plano de implementação em 12 lotes

Os lotes abaixo foram organizados para entregar valor progressivamente. Ao final dos primeiros lotes já haverá smoke tests úteis; os lotes seguintes aumentam profundidade, diagnóstico, escala e governança.

Lote 1 - Fundação e inventário E2E

Objetivo: Definir o padrão oficial e mapear a superfície real das 32 ferramentas.

Entregas:

- Inventário de rotas, formulários, ações, downloads e perfis de acesso.
- Convenções de nomenclatura e estrutura dos testes.
- Matriz de risco e prioridade por ferramenta.
- Definição de smoke, regressão, completo e exploratório.
- Documento de contrato E2E e critérios de aceite.

Critério de aceite: Todas as ferramentas classificadas e nenhuma área crítica sem dono ou estratégia de teste.

Lote 2 - Ambiente E2E isolado

Objetivo: Criar execução segura e reproduzível sem tocar dados normais.

Entregas:

- Arquivo de ambiente E2E sem segredos reais.
- Banco de dados isolado, migrations e seeds determinísticos.
- Usuários de teste para visitante, gratuito, Plus e administrador.
- Storage, mail, filas e integrações em modo fake ou sandbox.
- Comandos únicos para preparar, executar e limpar o ambiente.

Critério de aceite: Uma execução limpa pode ser repetida localmente com o mesmo resultado.

Lote 3 - Playwright e navegador base

Objetivo: Instalar o motor de navegação e criar a infraestrutura técnica inicial.

Entregas:

- Configuração Playwright e browsers.
- Base URL, timeouts, retries e execução paralela.
- Captura de screenshots, vídeos e traces.
- Hooks de autenticação e preparação de dados.
- Relatório HTML inicial.

Critério de aceite: Um teste piloto abre uma ferramenta, interage com ela e gera evidências.

Lote 4 - Contrato visual e seletores estáveis

Objetivo: Evitar testes frágeis dependentes de CSS, texto ou posição.

Entregas:

- Padrão data-testid para formulário, campos, submit, resultado, erros e downloads.
- Helpers Blade ou componentes compartilhados.
- Atributos aplicados aos layouts e componentes centrais.
- Validador arquitetural que identifica elementos obrigatórios ausentes.

Critério de aceite: Os testes críticos usam seletores estáveis e não dependem de classes Bootstrap.

Lote 5 - Descoberta automática e smoke universal

Objetivo: Testar automaticamente toda ferramenta registrada no catálogo.

Entregas:

- Exportação do catálogo para o runner E2E.
- Teste genérico de abertura e integridade.
- Validação de rota, título, formulário, submit e bloco principal.
- Captura automática de erros HTTP, JavaScript e console.
- Resumo por ferramenta.

Critério de aceite: As 32 ferramentas são descobertas sem lista manual e todas participam do smoke test.

Lote 6 - Motor declarativo de cenários

Objetivo: Permitir que cada módulo descreva seus dados e expectativas sem duplicar o motor.

Entregas:

- Schema PHP ou JSON para cenários.
- Executor de inputs, selects, checkboxes, datas e ações condicionais.
- Expectativas de texto, visibilidade, URL, estado e dados.
- Validação do schema e mensagens claras para cenário mal definido.
- Primeiras ferramentas piloto migradas.

Critério de aceite: Um mesmo executor roda cenários diferentes sem código de navegação duplicado.

Lote 7 - Instrumentação, correlação e logs

Objetivo: Transformar falhas em diagnósticos reproduzíveis.

Entregas:

- Middleware de correlação por execução e cenário.
- Canal JSON E2E.
- Captura de exceções, requests, responses, eventos e queries lentas.
- Integração dos logs com o relatório.
- Sanitização de dados sensíveis.

Critério de aceite: Toda falha de backend mostra cenário, requisição, exceção, arquivo, linha e logs relacionados.

Lote 8 - Validação profunda de downloads

Objetivo: Comprovar que cada botão gera um arquivo verdadeiro e utilizável.

Entregas:

- Interceptação e armazenamento de downloads.
- Validadores para PDF, XLSX, CSV, DOCX e ZIP.
- Validação de MIME, tamanho, assinatura e estrutura.
- Detecção de HTML de erro disfarçado.
- Relatório com arquivo e causa da invalidação.

Critério de aceite: Todo download declarado é aberto ou inspecionado estruturalmente, não apenas clicado.

Lote 9 - Acesso, autenticação e fluxos transversais

Objetivo: Cobrir comportamentos compartilhados e perfis comerciais.

Entregas:

- Sessões reutilizáveis por perfil.
- Testes de visitante, gratuito, Plus e administrador.
- Histórico, favoritos, persistência e exportação.
- Bloqueios na interface e nos endpoints.
- Throttling, CSRF e redirecionamentos essenciais.

Critério de aceite: Recursos protegidos falham ou funcionam corretamente em todos os perfis declarados.

Lote 10 - Cobertura das 32 ferramentas

Objetivo: Levar todas as ferramentas ao contrato mínimo de qualidade.

Entregas:

- Cenário válido principal para cada ferramenta.
- Cenário inválido essencial para cada ferramenta.
- Casos de borda conforme o risco.
- Downloads e ações secundárias declarados.
- Correção das falhas descobertas durante a implantação.

Critério de aceite: 100% das ferramentas possuem cobertura mínima e nenhuma depende exclusivamente de teste manual.

Lote 11 - CI, paralelismo e relatórios executivos

Objetivo: Tornar a automação parte do fluxo normal de desenvolvimento.

Entregas:

- Pipeline de commit, pull request e pré-release.
- Sharding por ferramenta e execução paralela.
- Cache seguro de dependências e browsers.
- Publicação de relatórios e artefatos.
- Comparação entre falhas novas, conhecidas e resolvidas.

Critério de aceite: Uma mudança com regressão bloqueia a integração e apresenta evidências para correção.

Lote 12 - Exploração, governança e manutenção contínua

Objetivo: Garantir que a solução continue útil quando o catálogo crescer.

Entregas:

- Modo exploratório/fuzzing de interface controlado.
- Responsividade e múltiplos navegadores na bateria completa.
- Regra arquitetural obrigando cenário E2E em ferramenta nova.
- Painel de cobertura, flakiness e duração.
- Política de manutenção, retenção e revisão de cenários.

Critério de aceite: Ferramentas novas entram automaticamente na automação e a saúde da suíte é mensurável.

10. Dependências entre os lotes

Faixa	Lotes	Resultado obtido
Fundação	1 a 4	Padrão, ambiente, Playwright e interface testável
Primeira automação útil	5 a 7	Smoke universal, cenários e diagnóstico correlacionado
Cobertura funcional	8 a 10	Downloads, acessos e cobertura das 32 ferramentas
Escala e continuidade	11 e 12	CI, paralelismo, governança e crescimento contínuo

11. Critérios mínimos para considerar a automação pronta

- Todas as ferramentas são descobertas pelo catálogo, sem lista manual duplicada.
- Cada ferramenta possui ao menos um cenário válido e um inválido.
- Ferramentas com download têm ao menos um arquivo validado internamente.
- Falhas de frontend e backend aparecem correlacionadas no mesmo relatório.
- A execução não usa nem altera o banco de desenvolvimento ou produção.
- Os testes podem rodar em paralelo sem compartilhar estado indevido.
- A suíte rápida é estável e adequada para uso frequente.

- O pipeline completo é obrigatório antes de uma release.
- Nova ferramenta sem contrato E2E é rejeitada pelo validador arquitetural.

12. Riscos e controles

Risco	Impacto	Controle recomendado
Testes frágeis por alteração visual	Muitas falhas falsas	data-testid e page objects/helpers estáveis
Cenários sem resultado normativo confiável	Teste confirma comportamento incorreto	Casos dourados revisados e fontes aprovadas
Compartilhamento de banco entre testes	Flakiness e contaminação	Banco isolado, transactions ou reset determinístico
Logs com dados sensíveis	Exposição de informações	Sanitização, allowlist de campos e retenção curta
Suíte completa lenta	Equipe deixa de executar	Perfis de execução, sharding e paralelismo
Integrações externas instáveis	Falhas fora do controle do projeto	Fakes, sandboxes e contract tests separados
Falsos positivos no modo exploratório	Ruído no relatório	Separar regressão determinística de exploração
Vídeos e traces muito grandes	Custo de armazenamento	Guardar evidências principalmente nas falhas

13. Itens que não devem ser feitos

- Inserir e remover Log::info automaticamente nos arquivos de produção.
- Usar o banco real de desenvolvimento ou produção nos testes E2E.
- Depender apenas de textos visíveis ou classes CSS como seletores.
- Aceitar que um botão de download está correto somente porque houve um clique.
- Criar um script totalmente diferente para cada ferramenta.
- Misturar testes determinísticos com fuzzing no mesmo critério de bloqueio.
- Registrar senhas, tokens, documentos completos ou dados pessoais nos artefatos.

14. Estrutura de pastas sugerida

```
tests/
├── Browser/
│   ├── fixtures/
│   ├── helpers/
│   ├── pages/
│   └── playwright/
│       ├── tool-smoke.spec.ts
│       ├── tool-scenarios.spec.ts
│       ├── tool-downloads.spec.ts
│       ├── tool-access.spec.ts
│       └── tool-responsive.spec.ts
├── Feature/
├── Unit/
└── Architecture/
```

```
app/Core/Quality/E2E/
├── Contracts/
```

- └─ Middleware/
- └─ Logging/
- └─ Reporting/
- └─ ToolScenarioRegistry.php
- └─ ToolScenario.php
- └─ ToolDownloadExpectation.php

storage/app/e2e-artifacts/

15. Exemplo do relatório de falha

Ferramenta: VacationCalculator

Cenário: férias com venda de 10 dias

Status: FALHOU

Etapas: clique em Calcular

Dados:

salary=3500,00; vacation_days=30; sold_days=10; dependents=2

Frontend:

Nenhum resultado apareceu após o tempo limite.

Console: Failed to load resource - HTTP 500

Backend:

POST /ferramentas/calculo-ferias - HTTP 500

Exceção: DivisionByZeroError

Arquivo: VacationCalculationService.php

Linha: 148

Evidências:

screenshot.png

trace.zip

laravel-correlated.json

request-response.json

16. Recomendação final

A automação deve ser tratada como uma capacidade permanente do Core de Qualidade. A primeira meta prática é chegar ao fim do Lote 7, quando já existirão descoberta automática, smoke tests das 32 ferramentas, cenários declarativos e diagnóstico correlacionado. Depois, os Lotes 8 a 10 concluem a cobertura funcional atual; os Lotes 11 e 12 garantem escala e continuidade.

Quantidade total recomendada: 12 lotes

Essa divisão reduz risco, permite validar a arquitetura por etapas e evita tentar automatizar as 32 ferramentas antes de existir um motor estável. O objetivo final é que a manutenção da suíte cresça de forma muito mais lenta do que a quantidade de ferramentas.